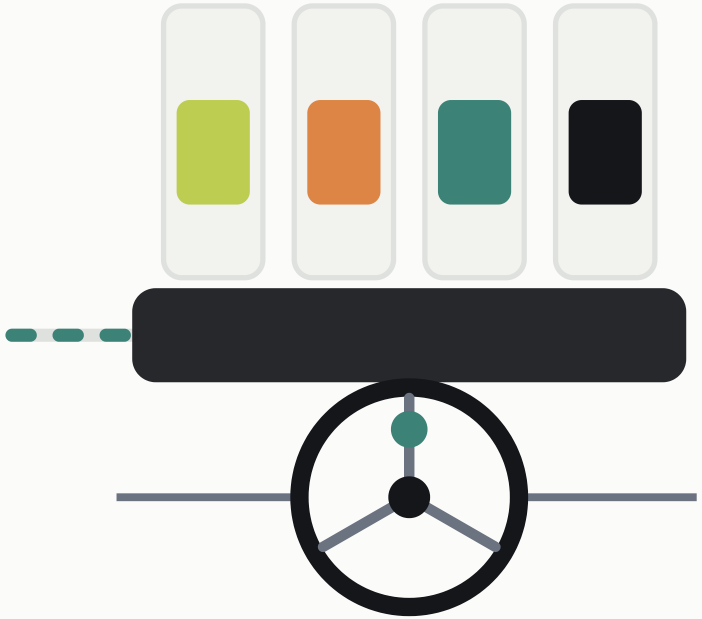
The New Engine for Bayesian Inference with INLA

Structure-adaptive solver for the linear algebra inside every INLA fit.



Esmail Abdul Fattah

Joint work with Hatem Ltaief, David E. Keyes and Håvard Rue

King Abdullah University of Science and Technology

8.9 seconds



Not only him

5,695
open-access papers use INLA

→

1,071
say how long it took, or compare it

→

168
print the benchmark table

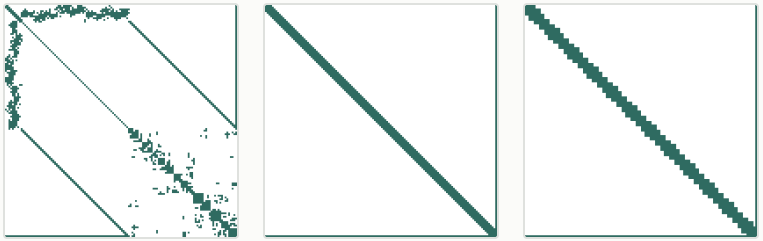
Open-access full text only, screened for papers that genuinely use INLA, so every count here is a floor.

Nearly one in five raises the time it takes.

Why it is fast, and why it spread

Sparsity

Fewer than **1 in 1,000** entries of the precision matrix $Q(\theta)$ is non-zero, so the factorization never touches the rest.



n = 1,174 n = 14,401 n = 26,113

Generality

One latent field, any likelihood, and the same call shape over the lot of them.

114

likelihood families, 52 latent models, 48 priors

+ your own

write your own likelihood or latent effect and it meets the same call: the catalogue is where you start, not where you stop

Portability

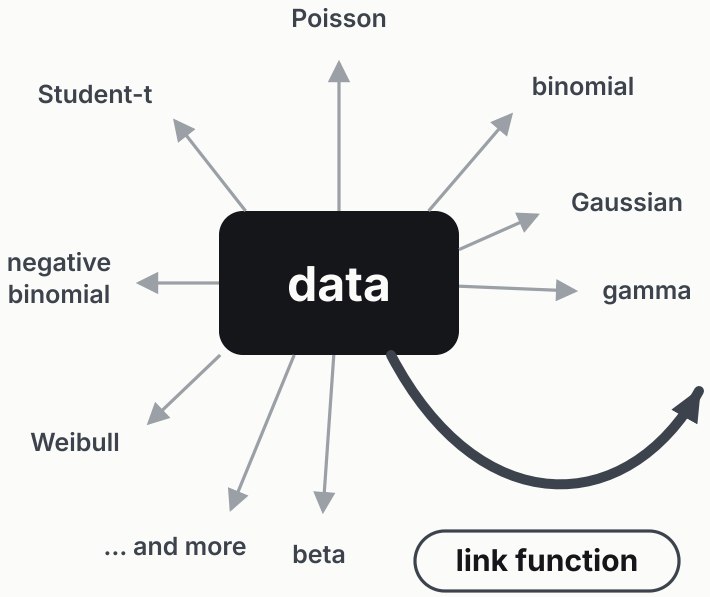
The model is one line of R, and it runs on whatever the room happens to own.

Linux · macOS · Windows

binaries ship in the package, nothing to compile

Since **2009**, and **186** papers that extend the method rather than just use it.

What the data is made of



$$y \mid x, \theta \sim \prod_i \pi(y_i \mid x_i, \theta)$$

$$x \mid \theta \sim \mathcal{N}(0,\ Q(\theta)^{-1})$$



covariate
a straight effect

no hyperparameter

iid
unstructured

time
a smooth trend

space
a smooth field

interaction
the field moves

more effects
as needed

hyperparameters
how strong, how smooth, how far each effect reaches

$$\theta \sim \pi(\theta)$$

a handful of numbers, and they set everything above

The assumptions of 2009

1. The precision matrix Q is sparse
2. A handful of hyperparameters
3. The latent field is Gaussian

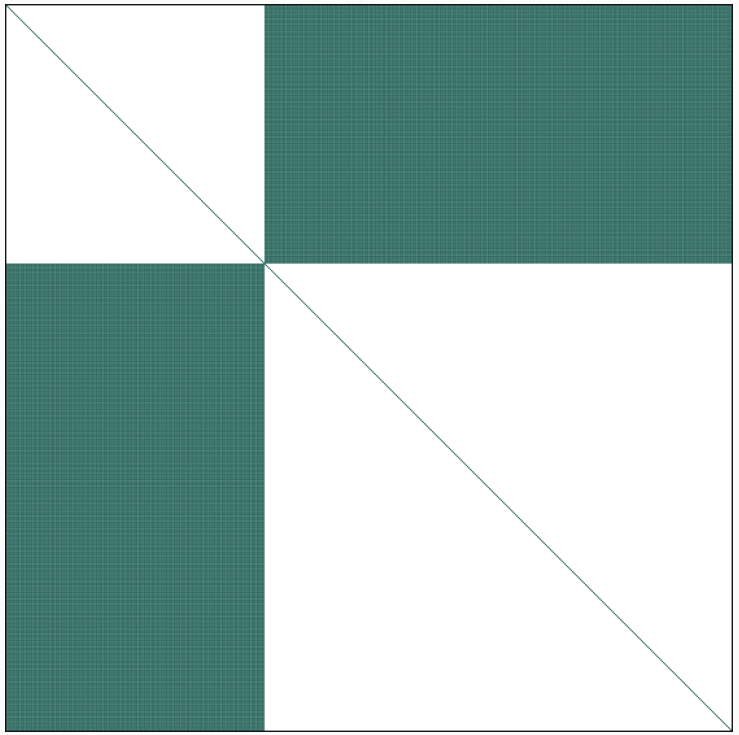
The assumptions of 2009

The precision matrix *Q* is sparse?

```
y ~ 1
  + f(subject, "iid")
  + f(item, "iid")
```

200 SUBJECTS, 360 ITEMS · EVERY SUBJECT SEES EVERY ITEM

561
unknowns



Q, as INLA assembled it

Why INLA needs HPC

Practice has outgrown the assumptions of 2009.

620,000

`inla()` calls for **one** trial design, took 20 days![1]

2.4 days

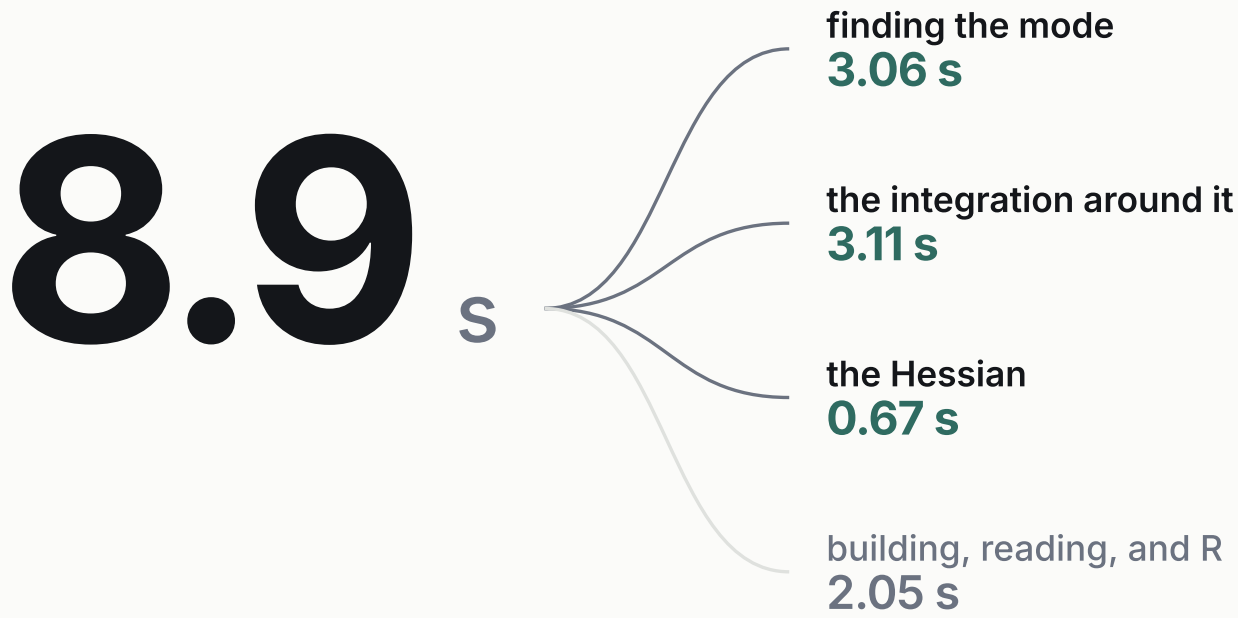
for **one** model, on 10 threads and 64 GB[2]

1 million

latent parameters in **one** model, and 2 million observations[3]

1 Hosseini et al., *Contemp. Clin. Trials* **142**, 2024 · 2 Quijal-Zamorano et al., *Int. J. Epidemiol.* **54**(4), 2025 · 3 Gaedke-Merzhäuser et al., *SIAM J. Sci. Comput.* **46**(4), 2024

What those seconds are



1	41	11	8,705
ORDERING	EVALUATIONS	SELECTED INVERSES	POSTERIOR MARGINALS

$Q(\theta)\,x = b$

a factorization, then a solve · **every evaluation**

$\operatorname{diag} Q(\theta)^{-1}$

a selected inverse · **every configuration**

Not one of these is the model. All of it is linear algebra on $Q(\theta)$.

To make it faster

1 **A faster solve**

$Q(\theta)\,x = b$, which is the factorization $Q = LL^{\mathsf{T}}$ under it

2 **A faster selected inverse**

$\operatorname{diag} Q(\theta)^{-1}$, once per configuration

3 **Memory known, and more of the machine**

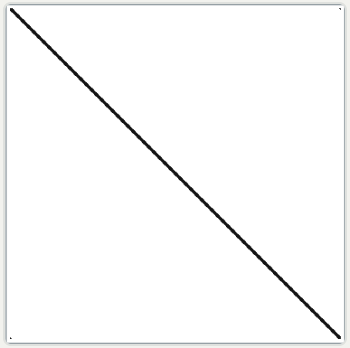
what the factor will cost, and how the parallelism should be tuned

sTiles

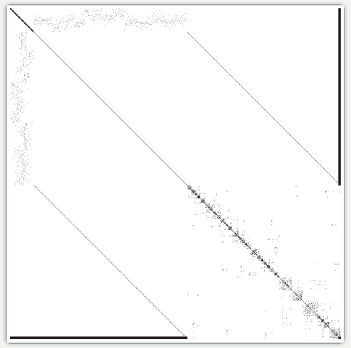


A high-performance framework for structured matrices

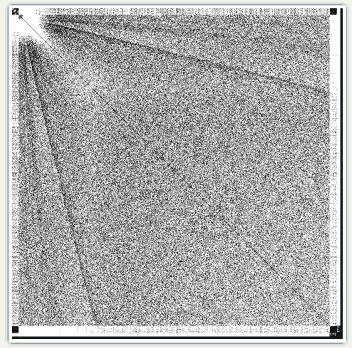
The first solver that covers the full range,
from **very sparse** to **fully dense**



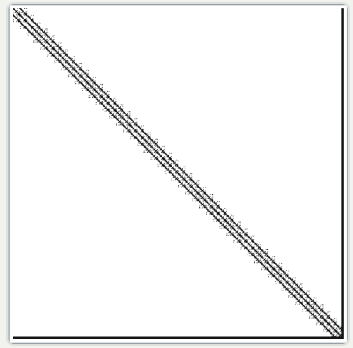
a smooth in time
rw2 · Tokyo · n = 366



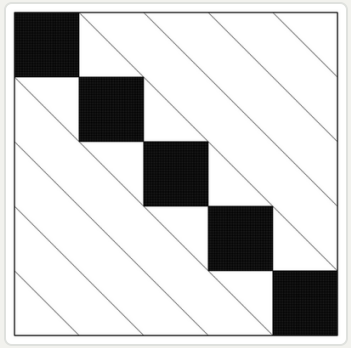
a map of districts
bym2 · Germany · n = 1,174



continuous space
spde mesh · n = 24,455



space times time
48 periods · n = 26,113



a pedigree of animals
animal1 · n = 5,126 · 20%



all pairs of points
kriging · dense precision

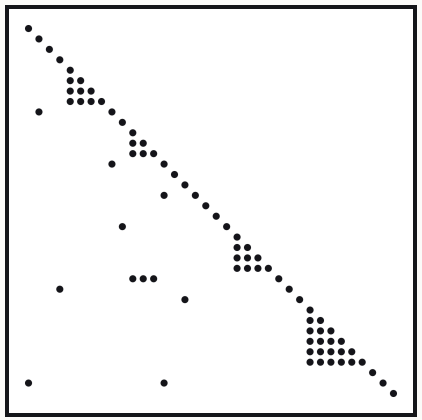
different sparsity patterns → fully dense

One methodology, a few lines of R. Underneath it the matrix goes from banded to graph to mesh to block to fully dense, and **no single representation is right for all of them**.

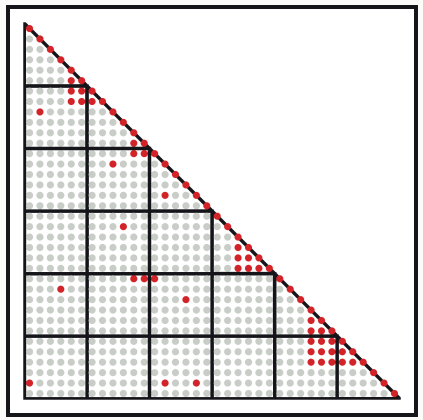
How much of the tile do you keep?

a nonzero a stored zero | one matrix, 36 × 36

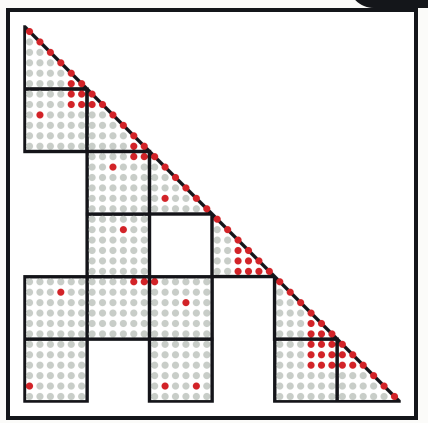
sTiles



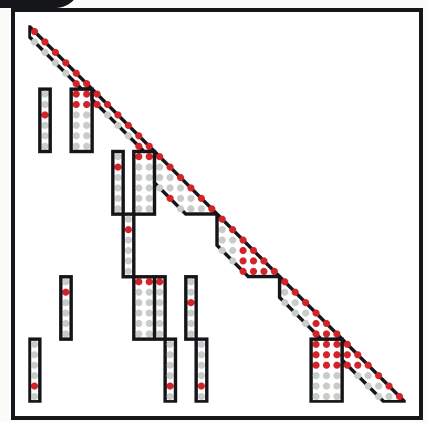
the matrix



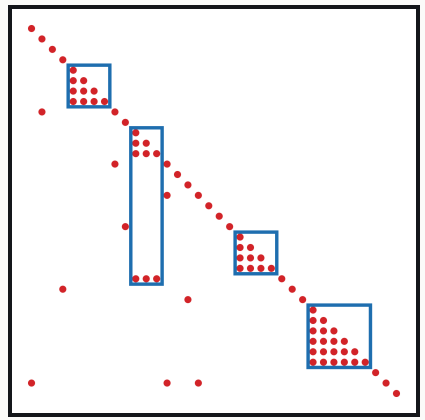
full dense



dense



semisparse



sparse

the matrix decides which one wins · **sTiles picks it**

The assumptions, revisited

1 ~~The precision matrix Q is sparse~~

2 ~~A handful of hyperparameters~~

More than a handful, and each one multiplies the solves

the design table runs out at **52** of them, **4,201** points, and after that the points have to be your own.

3 The latent field is Gaussian

It still is, and the extensions need it to be

What limits it now

the hyperparameters

how many evaluations the integration asks for

the parallel scheme

how many of them you hold at once

the structure of *Q*

how big one factor is, after fill-in

↓

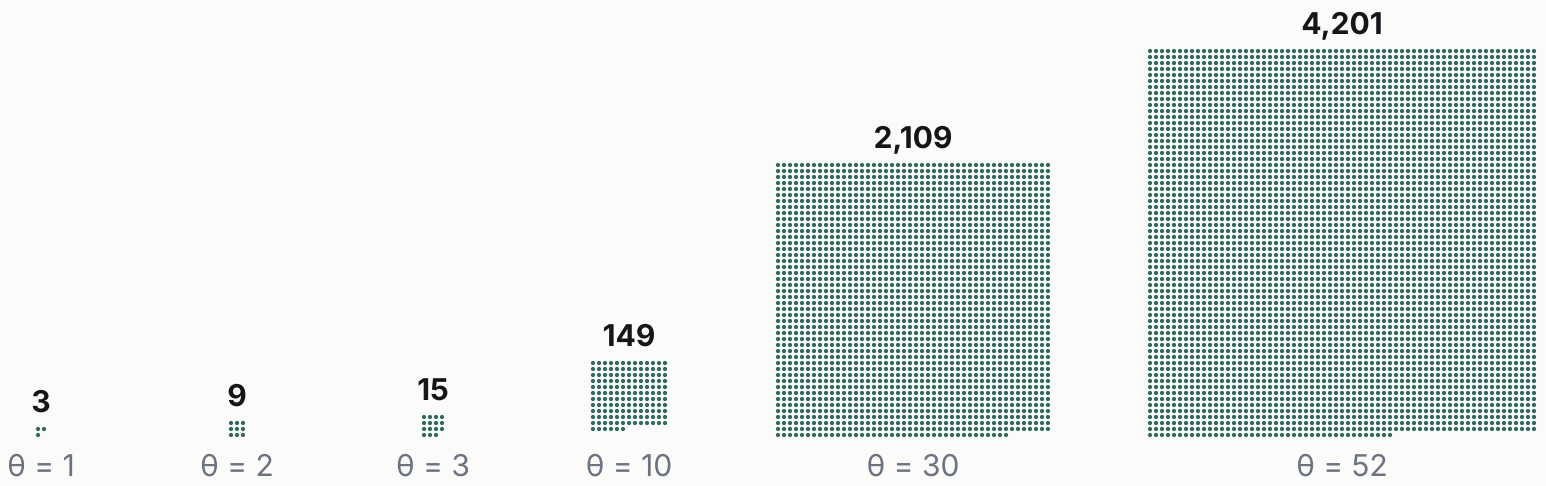
Does it fit in memory?

memory grows with **evaluations in flight** × **nnz(L)**

Not one of the three is a property of the method.

The limit moved from INLA to the machine you run it on.

How many can run at once

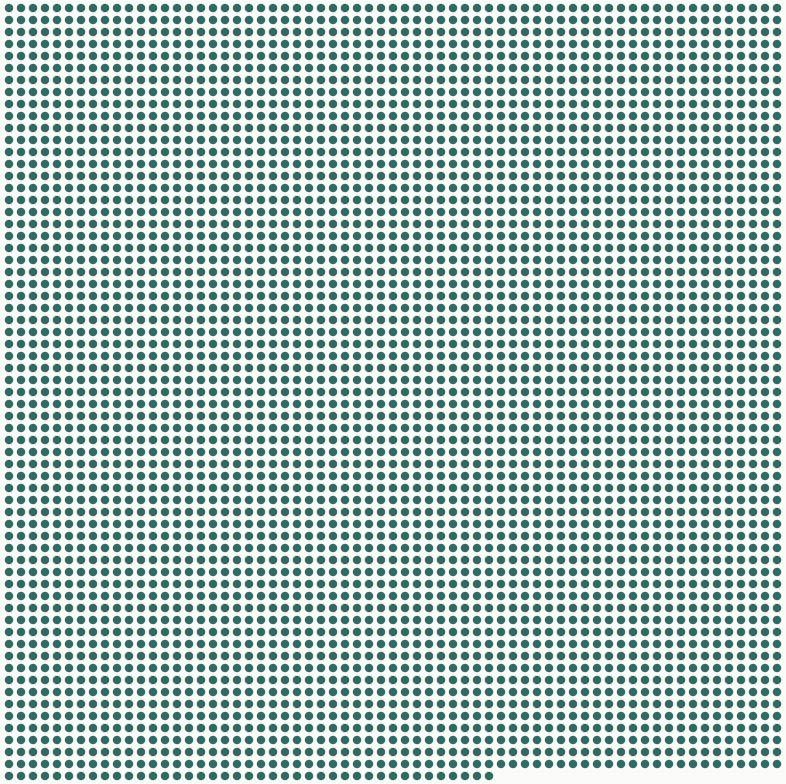


How many you run at once is **not** what the table allows.
It is what your machine has, divided by what **one evaluation** needs.

one factor in memory per evaluation in flight

What runs them at once

4,201



θ = 52 · one dot, **one evaluation**

DALIA

- on **GPUs**, through CuPy
- across **many nodes**, through MPI4Py
- in **Python**, modular and portable

github.com/dalia-project/DALIA

An application of the new engine

Modelling global temperature data for one year



A global temperature model

$\eta(s,t)$ is what one station s should read on one day t: the average of that day's low and high

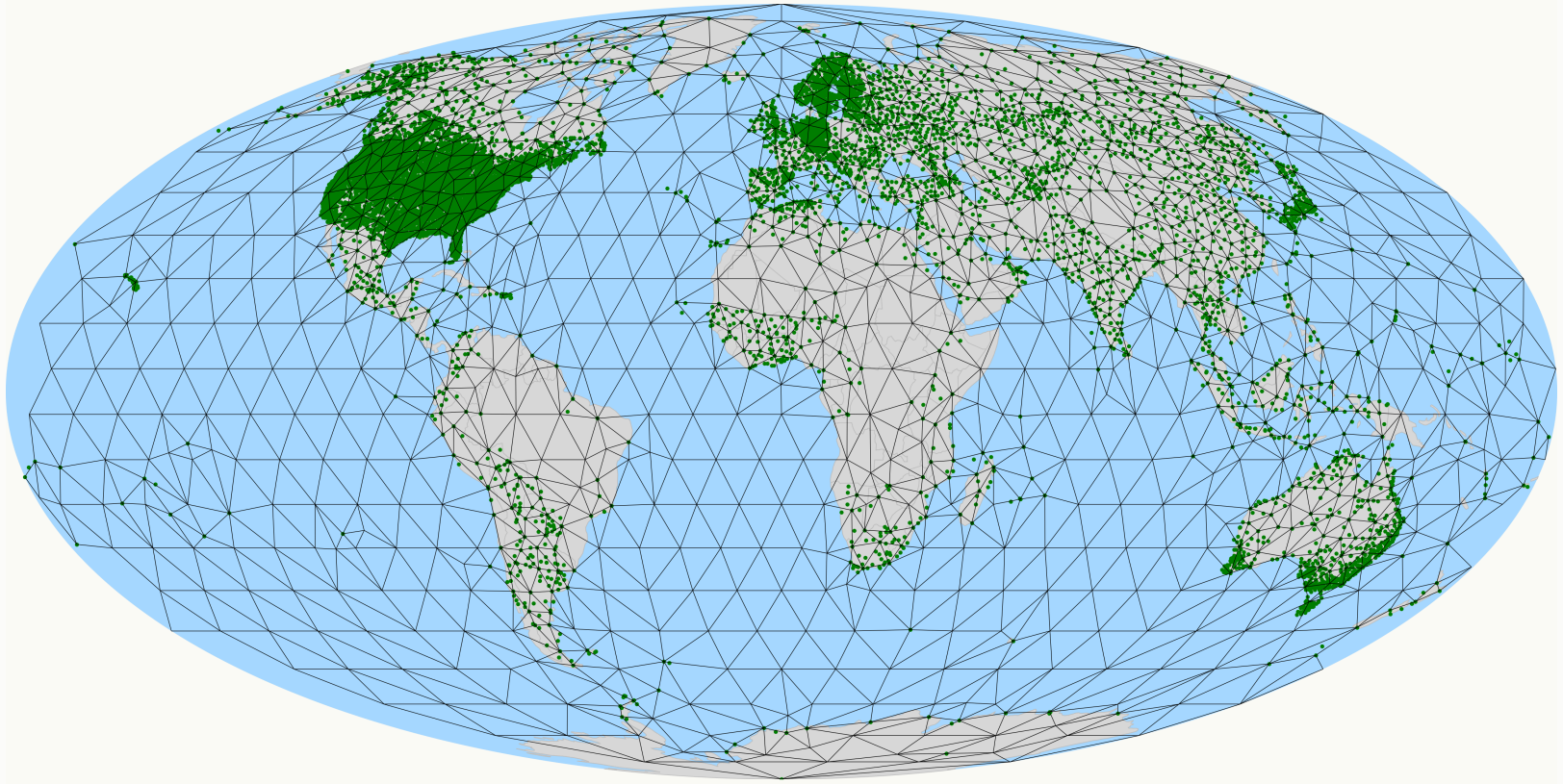
$$\eta(s,t) = \mu + \alpha E(s) + b(s,t) + v(s,t) + u(s,t)$$

$\mu + \alpha E(s)$	$b(s,t)$	$v(s,t)$	$u(s,t)$
one average for the globe, and how much colder per km of height	how it varies with latitude, and through the year	how one place departs from that pattern	the weather: space and time, coupled
2	**6**	**3,750**	**456,250**
numbers	basis columns	3 fields on the mesh	1,250 nodes × 365 days

460,008 unknowns, from **4,234,989** daily observations

Lindgren, Bakka, Bolin, Krainski and Rue, A diffusion-based spatio-temporal extension of Gaussian Matérn fields, *SORT* **48**(1), 2024

The spatial mesh, and where the data actually is



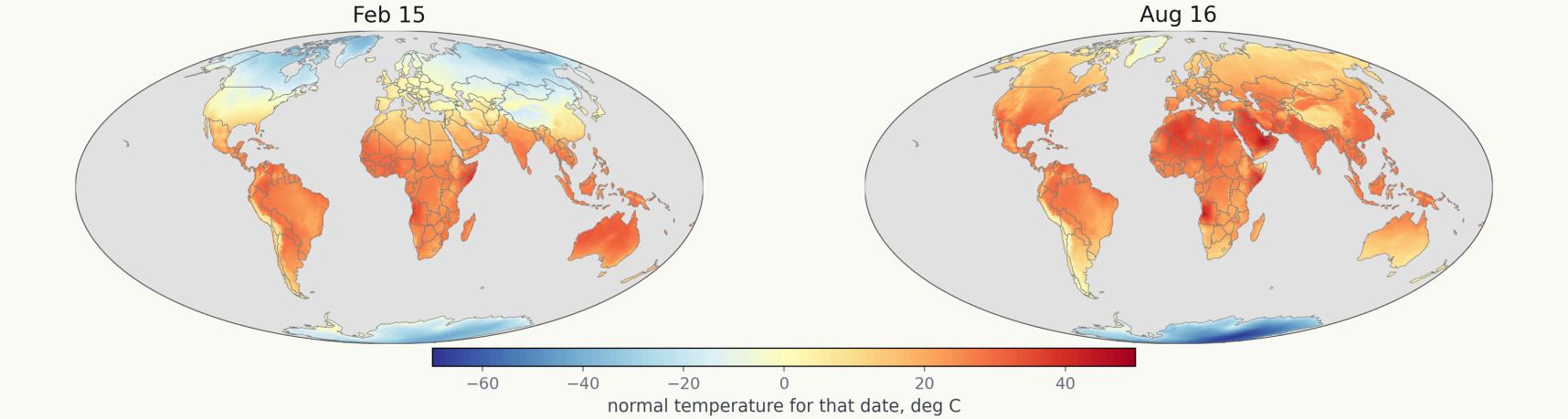
1,250
mesh nodes, 2,496 triangles
median edge 584 km

13,579
stations, 2022
4,234,989 daily observations

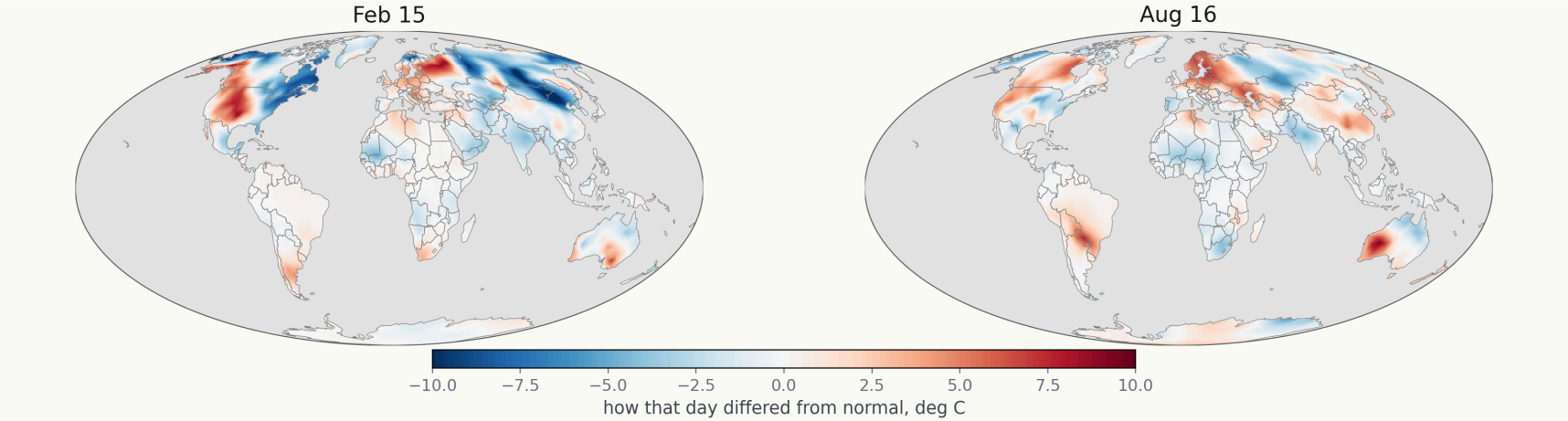
77.5%
of stations lie
between 30°N and 60°N

55.9%
of the whole network
is in the United States

Climate, and the weather on top of it



the normal for 15 February and 16 August



the same two days, as departures from those normals

Climate is what we **expect**; weather is what we **get**.

What one fit costs

Model	INLA 2024 · PARDISO	INLA 2026 · sTiles
102	7 h 07 m	**1 h 19 m**
121	21 h 05 m	**1 h 34 m**
202	8 h 35 m	**1 h 38 m**
220	31 h 27 m	**4 h 19 m**
all four	**68 h 14 m**	**8 h 50 m**

460,008 unknowns, 4,234,989 observations, every evaluation a factorization. **Three of the four fit in under two hours.**

55 INLA-based packages can run on sTiles

5
REGRESSION, TEACHING
qbrms · powerbrmsINLA · gdpar · abn · vibass

5
ECOLOGY, GENETICS
PointedSDMs · intSDM · SimSurvey · phyr · plantmix

4
SURVEYS
SUMMER · surveyPrev · sae4health · jollofR

3
NEUROIMAGING
BayesfMRI · BayesBrainMap · templateICAr

7
TRIALS, HEALTH ECONOMICS
BATSS · CRTspat · meta4diag · nmaINLA · survHE · blendR · voi

12
DISEASE MAPPING
bigDM · diseasemapping · DClusterm · eCAR · GHRmodel · SpatGC · sfclust · bdlnm · climatehealth · MAPCtools · nowcaster · surveillance

8
GEOSTATISTICS
geostatsp · bmstdr · mbg · MBGapp · midasINLA · AGPRIS · SpatialGEV · highriskzone

11
EXTENSIONS OF INLA
inlabru · INLAjoint · INLABMA · INLAcircular · inlamemi · excursions · rSPDE · MetricGraph · ngme2 · graphpcor · makemyprior

Each of them fits through `inla()`. **A faster engine is faster for all of them.**

Thank you

sTiles



- **Benchmarks**
- **Examples**
  Python · R · Julia · C++
- **Runs on**
  CPU · GPU
- **Install**
  Linux · macOS · Windows